

Uncertainty-aware Low-Rank Q -Matrix Estimation for Deep Reinforcement Learning

Tong Sang, Hongyao Tang, Jianye Hao*, Yan Zheng*, and Zhaopeng Meng

College of Intelligence and Computing, Tianjin University, China
 {2019218044,bluecontra,jianye.hao,yanzheng,mengzp}@tju.edu.cn

Abstract. Value estimation is one key problem in Reinforcement Learning. Albeit many successes have been achieved by Deep Reinforcement Learning (DRL) in different fields, the underlying structure and learning dynamics of value function, especially with complex function approximation, are not fully understood. In this paper, we report that decreasing rank of Q -matrix widely exists during learning process across a series of continuous control tasks for different popular algorithms. We hypothesize that the low-rank phenomenon indicates the common learning dynamics of Q -matrix from stochastic high dimensional space to smooth low dimensional space. Moreover, we reveal a positive correlation between value matrix rank and value estimation uncertainty. Inspired by above evidence, we propose a novel **Uncertainty-Aware Low-rank Q -matrix Estimation (UA-LQE)** algorithm as a general framework to facilitate the learning of value function. Through quantifying the uncertainty of state-action value estimation, we selectively erase the entries of highly uncertain values in state-action value matrix and conduct low-rank matrix reconstruction for them to recover their values. Such a reconstruction exploits the underlying structure of value matrix to improve the value approximation, thus leading to a more efficient learning process of value function. In the experiments, we evaluate the efficacy of UA-LQE in several representative OpenAI MuJoCo continuous control tasks.

Keywords: Reinforcement Learning · Value Estimation · Uncertainty · Low Rank.

1 Introduction

In recent years, Deep Reinforcement Learning (DRL) has been demonstrated to be a promising approach to solve complex sequential decision-making problems in different domains [11,16,22,26,6,21]. Albeit the progress made in RL, the intriguing learning dynamics is still not well known, especially when with complex function approximation, e.g., deep neural network (DNN). The little understanding of how RL performs and proceeds prevents the deep improvement and utilization of RL algorithms.

* Jianye Hao and Yan Zheng are the corresponding authors.

This paper is accepted by The 3rd International Conference on Distributed Artificial Intelligence (DAI 2021, Shanghai, China).

To understand the learning dynamics of RL, value function is one of the most important functions to consider. Value function defines the expected cumulative rewards (i.e., returns) of a policy, indicating how a state or taking an action under a state could be beneficial when performing the policy. It plays a vital role in RL for both value-based methods (e.g., DQN [16]) and policy-based methods (e.g., DDPG [11]). The learning dynamics and underlying structure of value function can be easily investigated in tabular RL by classical RL algorithms [24], where typically the action-value function Q of finite states and actions is maintained as a table. Towards larger state-action space, linear function approximation is widely as a representative option used for analysis [20,13,8]. Nevertheless, with complex non-linear function approximation as DRL, the learning dynamics and underlying structure are not well understood. A few works provide some insights on this with different empirical discoveries [7,28,12,10].

One notable work is Structured Value-based RL (SVRL) [28]. Yang et al. [28] identifies the existence of low-rank Q -functions in both classical control with Dynamic Programming [17] and Atari games with DQN [16]. In specific, the approximate rank of Q -matrix spanned by (sampled) states and actions decreases during the learning process. This indicates an underlying low-rank structure of the optimal value function and the learning dynamics of rank decrease. By leveraging this low-rank structure of Q -function, SVRL performs random drop of Q -matrix entries and then matrix estimation (or matrix completion). The intuition behind SVRL is the utilization of global structure of Q -function to reduce the value estimation errors thus facilitates the value function learning. Despite of the promising results achieved by SVRL, the low-rank structure of value function has not been investigated in continuous control. In addition, the selection of Q -matrix entries is aimless, in other words, it does not take into account the quality of estimate values, thus the underlying information from the low-rank structure is not fully and accurately utilized.

In this paper, we aim at addressing the above two problems and propose a novel algorithm called **Uncertainty-Aware Low-rank Q -matrix Estimation (UA-LQE)**, as a general framework to facilitate the learning of value function of DRL agents. We first examine the low-rank structure and learning dynamics of Q -functions in continuous control task. A quick view of the conclusion is shown in Fig. 1, the rank of sampled Q -matrix of representative continuous control DRL algorithms (DDPG [11], TD3 [4] and SAC [5]) decreases during the learning process. Our empirical evidence completes the SVRL’s discovery in DRL, indicating the generality of low-rank structure of Q -function across different algorithms and environments. Thus, we can expect that an effective method that makes use of the low-rank structure to improve RL will be beneficial in RL community broadly. Moreover, we take a step further and empirically reveal a positive correlation between value matrix rank and value estimation uncertainty, i.e., a value matrix of high rank often has an high overall value estimation uncertainty. Thus, we hypothesize that value estimation uncertainty can be used as the indicator of the target entries in Q -value matrix. This is because that the value estimate with high uncertainty often has a large estimation error that may

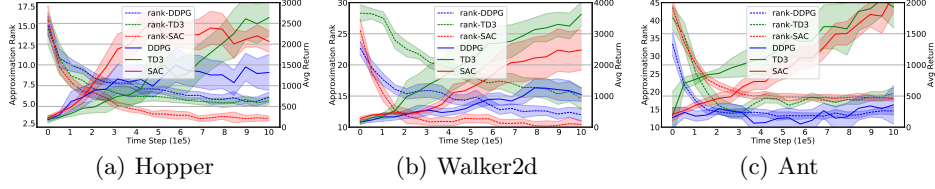


Fig. 1. Low-rank structure of Q -matrix in MuJoCo continuous control tasks. The left and right vertical axes denote the approximate rank of Q -matrix and average return. For all tasks and all algorithms, the rank decreases along the learning process. The results are averaged over 3 runs and the shaded region denotes a standard deviation.

severely violate the underlying structure of Q -value matrix, making themselves be the major ‘sinners’ to the induced high rank of value matrix. To this end, rather than erasing the entries and performing matrix estimation to reconstruct the value aimlessly as in SVRL, we quantify the uncertainty of state-action value estimates, then reconstruct the entries of high uncertainty in Q -value matrix. We expect the uncertainty-aware reconstruction to better reduce the value estimation errors and facilitate the emergence of the low-rank structured value function. In specific, we adopt deep ensemble-based and count-based uncertainty quantification and conduct value matrix reconstruction on both online value estimates (from evaluate network) and target value estimates (from target network). In the experiments, we evaluate the effectiveness of UA-LQE with DDPG [11] as the base algorithm in OpenAI MuJoCo continuous control tasks.

We summarize our contributions as follows: 1) We empirically demonstrate the existence of low-rank structure of Q -function for representative DRL algorithms in widely-adopted continuous control tasks. 2) We empirically reveals a positive correlation between value matrix rank and value estimation uncertainty. 3) We propose a novel algorithm, i.e., Uncertainty-Aware Low-rank Q -matrix Estimation (UA-LQE). We demonstrate the effectiveness of UA-LQE in improving learning performance in our experiments.

2 Background

2.1 Reinforcement Learning

Consider a Markov Decision Process (MDP) defined by the tuple $\langle S, A, P, R, \gamma, T \rangle$ with state space $S \in \mathbb{R}^{d_s}$, action space $A \in \mathbb{R}^{d_a}$, transition function $P : S \times A \times S \rightarrow [0, 1]$, reward function $R : S \times A \rightarrow \mathbb{R}$, discount factor $\gamma \in [0, 1]$ and horizon T . The agent interacts with the MDP by performing its policy $\pi : S \rightarrow A$. An RL agent aims at optimizing its policy to maximize the expected discounted cumulative reward $J(\pi) = \mathbb{E}_\pi[\sum_{t=0}^T \gamma^t r_t]$, where $s_0 \sim \rho_0(s_0)$ the initial state distribution, $a_t \sim \pi(s_t)$, $s_{t+1} \sim \mathcal{P}(s_{t+1} | s_t, a_t)$ and $r_t = R(s_t, a_t)$. The state-action value function Q^π is defined as the expected cumulative discounted reward: $Q^\pi(s, a) = \mathbb{E}_\pi[\sum_{t=0}^T \gamma^t r_t | s_0 = s, a_0 = a]$ for all $s, a \in S \times A$.

With function approximation, the agent maintains a parameterized policy π_ϕ , while Q^{π_ϕ} can be approximated by Q_θ with parameter θ typically through minimizing Temporal Difference loss [24]:

$$L(\theta) = \mathbb{E}_{s,a,r,s' \sim D} \left[Q_\theta(s, a) - \mathbb{E}_{a' \sim \pi_{\bar{\phi}}(s')} (r + \gamma Q_{\bar{\theta}}(s', a')) \right]^2. \quad (1)$$

The policy π_ϕ can be updated by taking the gradient of the objective $\nabla_\phi J(\pi_\phi)$, e.g., with the deterministic policy gradient (DPG) [23]:

$$\nabla_\phi J(\pi_\phi) = \mathbb{E}_{s \sim \rho^{\pi_\phi}} \left[\nabla_\phi \pi_\phi(s) \nabla_a Q^\pi(s, a)|_{a=\pi_\phi(s)} \right], \quad (2)$$

where ρ^{π_ϕ} is the discounted state distribution under policy π_ϕ . $\bar{\phi}, \bar{\theta}$ are the parameters of *target networks*, which are periodically or smoothly updated from the parameters (i.e., ϕ, θ) of *evaluation networks*.

2.2 Approximate Rank and Matrix Reconstruction

For a finite state space and a finite action space, we can view the corresponding Q -function as a Q -value matrix (shortly Q -matrix) of shape $|S| \times |A|$. The approximate rank [28,10] for a threshold δ is defined as $\text{arank}_\delta(Q) = \min\{k : \sum_{i=1}^k \sigma_i(Q) \geq (1 - \delta) \sum_{i=1}^d \sigma_i(Q)\}$, where $\{\sigma_i(Q)\}$ are the singular values of Q -value matrix in decreasing order, i.e., $\sigma_1 \geq \dots \geq \sigma_d \geq 0$. In this paper, we use $\delta = 0.01$ and omit the subscript for clarity. In other words, approximate rank is the first k singular values that capture more than 99% variance of all singular values. Further, for continuous (infinite) state and action space, we can define that $\text{arank}(Q) = \frac{1}{N} \sum_{j=1}^N \text{arank}(\tilde{Q}_j)$, i.e., the empirical average approximate rank of sampled Q -value submatrix $\{\tilde{Q}_i\}$, each of which corresponds to sampled state and action subspaces S_i, A_i .

If $\text{arank}(Q) \ll \min\{|S|, |A|\}$, we say a Q -value matrix (or Q -function) is *low-rank*. This means that the matrix can be *redundant* in the sense that the whole matrix can be reconstructed or completed with only some entries are known. In this paper, we consider Soft-Impute algorithm [15] for matrix reconstruction and more background about matrix estimation is provided in Appendix A. In the context of RL, we also view this as the underlying structure of Q -value matrix, of which we make use to improve value function learning in RL.

3 Low-Rank Q -Matrix in DRL

In this section, we first empirically investigate the rank of Q -matrix for representative DRL agents in continuous control tasks (Sec. 3.1), which complements prior studies in discrete-action control tasks. Then, we introduce two ways to leverage the low-rank structure for DRL continuous control (Sec. 3.2).

3.1 Empirical Study of Low-Rank Q -Matrix in MuJoCo

To investigate the low-rank structure of Q -matrix of DRL agents in continuous control, we choose three representative DRL agents: DDPG [11], TD3 [4], SAC

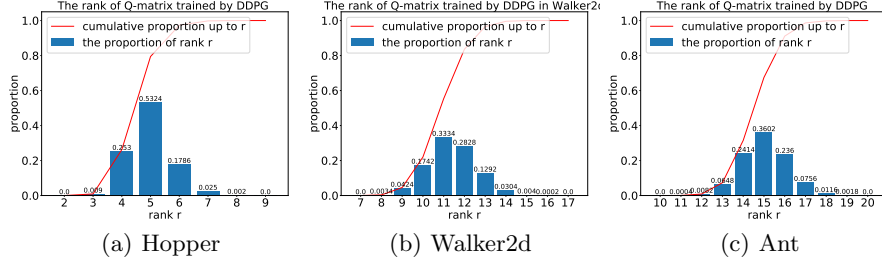


Fig. 2. Distribution of approximate rank of 10k sampled Q -matrix learned by DDPG agent after 1 million training on four MuJoCo environments.

[5], and we use MuJoCo environments as benchmarks. We train each algorithm in each environment for 1 million interaction steps; for each evaluation time point, we record the average return and the approximate rank by calculating the average of 10k sampled Q -matrix (i.e., $\{\tilde{Q}_i\}_{i=1}^{10000}$ with $|S_i| = |A_i| = 64$ for all i , sampled from replay buffer). In Fig. 1, we plot the curves of average return and approximate rank of learned Q network (i.e., $\text{arank}(Q_\theta)$). The results show that the rank of sampled Q -matrix of all three algorithms decreases during the learning process in all the environments. This reveals a general but intriguing underlying dynamics of value learning. Moreover, there shows no clear correlation between the rank and the average return by comparing across different algorithms. Intuitively, we hypothesize that such phenomena are caused by the integration of function approximation and TD backup, where the induced generalization and bootstrapping smooth the value estimates.

In Fig. 2, we statistic the distribution of the approximate rank of 10k sampled Q -matrix learned by DDPG. We can find that most Q -matrices concentrate at a low rank with the value about 2 to 3 times of the action dimensionality controlled. This shows that the (near) optimal Q -matrix can be low-rank. In addition, consider the extreme case that Q -values are independently influenced by each dimension of action and linear to action, the rank of Q -matrix should be the action dimensionality (i.e., $\dim(A)$) since each action’s value under some state can be represented by the values of $\dim(A)$ actions. The results indicate that the (near) optimal Q -matrix learned possesses a similarly simple relation between value and action as the extreme case.

3.2 Q -Matrix Reconstruction for DRL

The empirical results above show the dynamics of the approximate rank of Q -matrix, i.e., gradually decreasing during the learning process to a low rank. This reveals the smoothing nature of learning process and the underlying low-rank structure of the desired Q function. Based on similar empirical results obtained

For TD3 and SAC which adopt double critics $Q_{\theta_1}, Q_{\theta_2}$, we calculate the approximate rank for Q_{θ_1} since the two are equivalent.

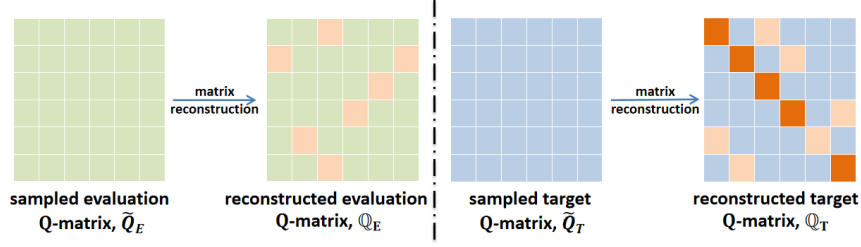


Fig. 3. Illustration of two ways of Q -matrix reconstruction. Consider a batch of transitions $\{(s, a, r, s')_i\}$ of size N sampled from replay buffer. (*Left*) For the sampled evaluation Q -matrix (green) $\tilde{Q}_E = \{Q_\theta(s, a)\}_{i,j=1,1}^{N,N}$, the entries (orange) selected (e.g., at random) are erased and then matrix estimation algorithm is performed to obtain the reconstructed Q_E , which is used to regularize the evaluation network. (*Right*) For the sampled target Q -matrix (blue) $\tilde{Q}_T = \{Q_\theta(s', a')\}_{i,j=1,1}^{N,N}$ where each $a' = \pi_{\bar{\phi}}(s')$, the entries are removed and re-estimated in the same way and the diagonal (red) is used as modified target values to approximate.

with discrete action space, Yang et al. [28] propose SVRL algorithm to improve conventional learning process, by conducting matrix reconstruction for target value matrix when performing TD learning of DQN [16]. In the following, we first make a simple extension of the matrix reconstruction for target value matrix to the continuous-action setting.

With discrete (and usually finite) action space, for each training of value function (recall Eq. 1), a mini-batch of N transitions $\{(s, a, r, s')_i\}$ are sampled from the replay buffer. SVRL [28] uses the N next states and all discrete actions to form the sampled target Q -matrix. While it is infeasible to use all actions from a continuous action space, we make use of the N next actions sampled by target policy $\pi_{\bar{\phi}}$ under corresponding N next states $\{(s', a')_i\}$ and form an $N \times N$ matrix, denoted by $\tilde{Q}_T = \{Q_{\bar{\theta}}(s', a')\}_{i,j}$. As in the left of Fig. 3, by randomly removing a portion of entries in $\tilde{Q}_T$ and then performing Soft-Impute matrix estimation, we can obtain the reconstructed Q -matrix denoted by Q_T , in which the removed entries are re-estimated by leveraging the underlying rank structure of Q -matrix. Then, we can obtain the modified TD value function learning with target values from Q_T as shown by the loss function below:

$$L_T(\theta) = \mathbb{E}_{s,a,r,s' \sim D} \left[Q_\theta(s, a) - \mathbb{E}_{a' \sim \pi_{\bar{\phi}}(s')} (r + \gamma Q_T(s', a')) \right]^2. \quad (3)$$

As proposed in SVRL [28], the reconstructed value estimates are expected and empirically demonstrated to be better and thus benefit the learning process. However, we can observe that only the diagonal value estimates in Q_T is used during updates; the reconstructed value estimates at randomly removed entries may not be used at all, thus being inefficient in leveraging the information of the underlying structure of Q -matrix.

To this end, we additionally propose matrix reconstruction on evaluate value matrix for the purpose of making more use of reconstructed value estimates. As in the right of Fig. 3, for a mini-batch of N transitions, we use the N state-action

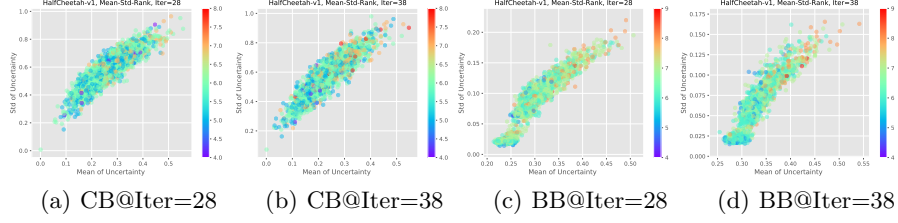


Fig. 4. Illustration of the positive correlation between approximate rank and value estimate uncertainty. We use DDPG agent on HalfCheetah for demonstration. Each point, corresponding to a sampled Q -matrix, is colored by its approximate rank with purple and red for the lowest and highest; and is placed on the 2D plane with the x-axis for the mean of its uncertainty matrix $U(\tilde{Q}_i)$ and the y-axis for the std. From (a)-(d), we plot the results in two different iterations for both count-based (CB) and bootstrapped-based (BB) uncertainty quantification methods.

pairs to form the sampled *evaluation* Q -matrix, denoted by $\tilde{Q}_E = \{Q_\theta(s, a)\}_{i,j}$. Then we can obtain the reconstructed $\mathbb{Q}_E$ and regularize Q network as follows:

$$L_E(\theta) = \mathbb{E}_{s,a \sim D} \left[\tilde{Q}_E(s, a) - \mathbb{Q}_E(s, a) \right]^2. \quad (4)$$

The above two ways improve value learning process differently. The former way provides superior target values to approximate by re-estimating values based on rank information involved in matrix reconstruction on target Q -matrix; while the latter works more like consistency regularization according to underlying rank structure.

4 DRL with Uncertainty-aware Q -Matrix Reconstruction

After the complementary study to prior work on empirical rank structure and reconstructed Q -matrix estimates in previous section, in this section, we take a further step to reveal the correlation between the approximate rank of Q -matrix and value estimation uncertainty. Based on this discovery, we propose an uncertainty-aware Q -matrix reconstruction method for DRL agents rather than the random removal and reconstruction.

4.1 Connection between Rank and Uncertainty

To investigate the relation between approximate rank of Q -matrix and value estimation uncertainty, we first introduce two techniques for uncertainty quantification: **count-based method** [1] and **bootstrapped-based method** [18].

Count-based (CB) method We denote the sampled times for training of a state-action pair by $N(s, a)$. For feasibility, original continuous state-action pairs are converted into discrete *Hash* codes [25] and thus similar pairs are counted together. We then use the inverse number of sampled times as our first measure of uncertainty [27]: $U_{CB}(s, a) = \frac{1}{N(s, a)}$.

Bootstrapped-based (BB) method Following previous works [18,3], we establish a bootstrapped ensemble of K independent value function estimators $\{Q_i\}_{i=1}^K$. For any state-action pair s, a , we obtain the uncertainty $U_{\text{BB}}(s, a)$ by calculating the standard deviation of the value estimates from the ensemble: $U_{\text{BB}}(s, a) = \sqrt{\frac{1}{N} \sum_{i=1}^N [Q_i(s, a) - \bar{Q}(s, a)]^2}$ where $\bar{Q}$ is the mean of the ensemble. This is also widely known as *epistemic uncertainty* in the literature of Bayesian exploration.

Next, with the above two uncertainty quantification of value estimates, we investigate the connection between the approximate rank of sampled Q -matrix and value estimate uncertainty. First, for some time point among the learning process, we sample 100 evaluation Q -matrix (similarly as done in Fig. 1 and 2). For each sampled Q -matrix $\tilde{Q}_i$, we obtain its approximate rank; we also quantify the value estimate uncertainty of each entry in $\tilde{Q}_i$, resulting in an uncertainty matrix $U(\tilde{Q}_i)$, and then calculate the mean and standard deviation (i.e., std for short) of $U(\tilde{Q}_i)$. In Fig. 4, we draw the scatter plots with the x-axis for the mean of each $U(\tilde{Q}_i)$ and the y-axis for the std. Each point in Fig. 4 denotes a sampled Q -matrix and is colored by its approximate rank (from purple to red). Similar results can also be observed for target Q -matrix and for other MuJoCo environments. Fig. 4 shows a positive correlation between the approximate rank of sampled Q -matrix and value estimate uncertainty. There also shows a slight trend that this correlation is more obvious as the training marches.

Intuitively, when the mean and std are both large, it means that there are many entries with high uncertainty in the Q -matrix. The highly inaccurate value estimates deviate from the underlying structure of true value matrix, thus leading to a high approximate rank. On the contrary, when the uncertainty is low, in other words the value estimates are relatively certain, the approximate rank is often low since the underlying structure is well approximated. Now, we reach the important insight that value estimate uncertainty that reflects value approximation error, can be the signifier in finding the target entries which deviate from the desired learning process and prevent the emergence of low-rank structure. Therefore, rather than performing at random, uncertainty-aware removal and reconstruction is aimless and thus can be more efficient in reducing the value approximation error and maintaining the underlying structure of Q -matrix, finally improving the learning process.

4.2 Uncertainty-aware Q -matrix Reconstruction for DRL

Based on the insight revealed in previous paragraph, in the following we proposed **Uncertainty-Aware Low-rank Q -matrix Estimation (UA-LQE)** algorithm.

The main idea of UA-LQE is to quantify the uncertainty of Q -value estimates during the learning process, and then perform Q -matrix reconstruction as introduced in Sec. 3.2 with re-estimated entries that are selected according to the quantified uncertainty. Let us take UA-LQE for *evaluation Q -matrix* reconstruction with bootstrapped-based uncertainty quantification for demonstration. Fig. 5 illustrates the process. We maintain an ensemble of K independent critic

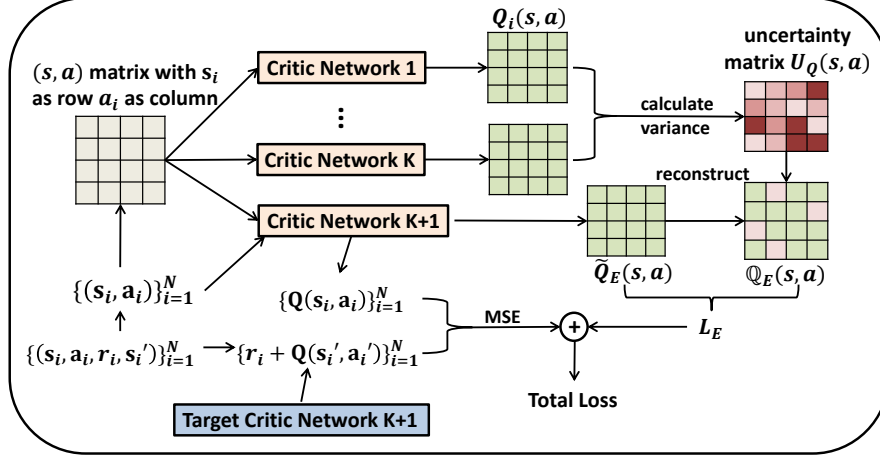


Fig. 5. Uncertainty-aware Q -matrix reconstruction based on bootstrapped-based uncertainty quantification. The bootstrapped ensemble of first K critic networks is used to calculate uncertainty, which is used for the selection of re-estimated entries in the Q -matrix. The reconstructed matrix is then used for the learning of the $K + 1$ critic network, i.e., the one for primal value estimation and policy learning. Only matrix reconstruction on sampled evaluation Q -matrix Q_E is plot for demonstration.

networks $\{Q_i\}_{i=1}^K$ (i.e., Q -function approximator) parallel to the major critic network Q_{K+1} . The critic networks in the ensemble are trained in convention according to Eq. 1 during the learning process. For each time of update for the major critic network, we can obtain the uncertainty matrix of sampled evaluation Q -matrix through the ensemble, i.e., $U_{BB}(\tilde{Q}_E)$. According to $U_{BB}(\tilde{Q}_E)$, we select the entries of **top p -percent highest uncertainty** for each row (corresponding to each sampled state) of $\tilde{Q}_E$ to remove; and then we obtain reconstructed evaluation Q -matrix Q_E and conduct the training as Eq. 4. The processes for target Q -matrix reconstruction and for count-based uncertainty quantification are similar which are omitted in Fig. 3 for clarity.

Next, we propose a practical implementation of DRL algorithm for continuous control by integrating DDPG as a base algorithm and UA-LQE. Note that UA-LQE is compatible with most off-the-shelf DRL algorithms. We use DDPG for an representative implementation and leave the other potential combination for future work. A detailed pseudo-code of DDPG with UA-LQE, with algorithmic options of sampled Q -matrix reconstruction and uncertainty quantification methods, is provided in Algorithm 1.

5 Experiment

In the experiment, we evaluate the efficacy of UA-LQE in several representative MuJoCo continuous control environments.

Algorithm 1: DDPG with Uncertainty-aware Low-Rank Q -Matrix Estimation (**DDPG with UA-LQE**)

```

1 Initialize actor  $\pi_\phi$  and critic networks  $Q_\theta$  with random parameters  $\phi, \theta$ , and
  target actor  $\pi_{\bar{\phi}}$  and critic networks  $Q_{\bar{\theta}}$  with  $\bar{\theta} = \theta, \bar{\phi} = \phi$ 
2 Initialize uncertainty estimator  $U_\omega(\cdot)$  with parameter  $\omega$  (i.e., random Hash
  table for CB and ensemble of critics for BB) and re-estimation percentage  $p$ 
3 Initialize replay buffer  $\mathcal{D}$ , loss weight  $\beta$ 
4 for Episode  $e \leftarrow 1$  to Max Episode Number do
5   Obtain the initial state  $s_0$ 
6   for Timestep  $t \leftarrow 1$  to  $T$  do
7     Select action  $a_t \sim \pi_\phi(s_t) + \epsilon_e$ , with exploration noise  $\epsilon_e \sim \mathcal{N}(0, \sigma)$ 
8     Execute  $a_t$  and observe reward  $r_t$  and new state  $s_{t+1}$ 
9     Store  $\{s_t, a_t, r_t, s_{t+1}\}$  in  $\mathcal{D}$ 
10    Sample a mini-batch of experience  $\Omega = \{s_i, a_i, r_i, s_{i+1}\}_{i=1}^N$  from  $\mathcal{D}$ 
11    // perform UA-LQE with algorithmic options (refer to Sec. 3.2)
12    if using Target  $Q$ -matrix Reconstruction: then
13      Calculate conventional TD loss  $L(\theta)$  as critic loss ▷ see Eq. 1
14    else
15      Establish evaluation  $Q$ -matrix  $\tilde{Q}_T$ , remove  $p$ -percentage entries
        based on  $U_\omega(\tilde{Q}_T)$  and obtain the reconstructed  $Q$ -matrix  $\mathbb{Q}_T$ 
16      Calculate  $L_T(\theta)$  as critic loss ▷ see Eq. 3
17    if using Evaluation  $Q$ -matrix Reconstruction: then
18      Establish evaluation  $Q$ -matrix  $\tilde{Q}_E$ , remove  $p$ -percentage entries
        based on  $U_\omega(\tilde{Q}_E)$  and obtain the reconstructed  $Q$ -matrix  $\mathbb{Q}_E$ 
19      Calculate  $L_E(\theta)$ , weight it by  $\beta$  and add to critic loss ▷ see Eq. 4
20    Update  $Q_\theta$  according to the total critic loss calculated above
21    Update  $\pi_\phi$  with deterministic policy gradient ▷ see Eq. 2
22    // update uncertainty estimator (refer to Sec. 4.1)
23    Update  $\omega$  accordingly, i.e., update the counts in Hash table for CB and
      update each critic in the ensemble with  $L(\theta_i)$  for BB ▷ see Eq. 1

```

5.1 Experiment Setup

We use four MuJoCo environments: *Hopper*, *Walker2d*, *Ant*, *HalfCheetah*, for our experimental evaluation. We run each algorithm (introduced in the next subsection) for 1 million steps for all four environments, and evaluate the performance of trained agents every 5000 steps. For each configuration, we run three independent trials with random environmental seeds and network initialization.

Baselines and Variants We provide an overview of the properties of the baseline and variants we considered in our experiments in Tab. 1. We use DDPG [11] as our base algorithm for the implementation of UA-LQE. We also extend prior algorithm SVRL [28] also based on DDPG for continuous control. With SVRL, we can evaluate the efficacy of uncertainty-aware Q -matrix reconstruction compared with at random. We also evaluate several UA-LQE variants of different

Table 1. Properties of baselines and variants evaluated in our experiments.
All the algorithms share the DDPG base. N.A. is short for ‘not applicable’.

Alg. / Prop.	Reconstruction	Rec. Loss	Selection	Uncertainty ($U_\omega(\cdot)$)
DDPG	✗	N.A.	N.A.	N.A.
SVRL-E	✓	$L_E(\theta)$	Random	N.A.
UA-LQE-E-CB	✓	$L_E(\theta)$	Uncertainty-aware	Count-based
UA-LQE-E-BB	✓	$L_E(\theta)$	Uncertainty-aware	Bootstrapped-based
SVRL-T	✓	$L_T(\theta)$	Random	N.A.
UA-LQE-T-CB	✓	$L_T(\theta)$	Uncertainty-aware	Count-based
UA-LQE-T-BB	✓	$L_T(\theta)$	Uncertainty-aware	Bootstrapped-based

algorithmic options from the perspectives of Q -matrix reconstruction losses (i.e., **E** or **T**) and uncertainty quantification methods (i.e., **CB** or **BB**).

Structure and Hyperparameters For all algorithms, we use two-layer fully connected networks with 200 nodes and ReLU activation (not including input and output layers) for both actor and critic networks of DDPG base structure. The learning rate is 0.0001 for actor and 0.001 for critic. We use a discount factor $\gamma = 0.99$. The parameters of target networks $(\bar{\theta}, \bar{\phi})$ is softly replaced with the ratio 0.001. After each episode ends, the actor and critic are trained for T times where T here denotes the number of time steps of the episode. The batch size is set to 64 for both actor and critic mini-batch training and sampled Q -matrix (i.e., with the size of 64×64). The uncertainty-aware re-estimation percentage is set to $p = 20$. For count-based (CB) method, we use 8-byte Hash codes [25] for each state-action pair. For bootstrapped-based (BB) method, we maintain an ensemble of $K = 10$ critic networks. For UA-LQE on evaluation Q -matrix, we weight $L_E(\theta)$ by $\beta = 0.1$ when it is added to the total loss.

5.2 Results and Analysis

The evaluation results of algorithms are shown in Tab. 2. Complete learning curves are provided in Appendix C. We conclude the results from different angles of comparison: **1)** Compared with DDPG baseline, all variants with Q -matrix reconstruction (both SVRL and UA-LQE) show an overall better performance (except for some variants in specific environments), demonstrating the efficacy of Q -matrix reconstruction in improving value estimation and thus the entire learning process. **2)** UA-LQE variants outperforms SVRL baselines significantly in Walker2d and Ant and show comparable results in HalfCheetah and Hopper. This shows the effectiveness of uncertainty-aware target entry selection during the process of Q -matrix reconstruction. This reflects the intuition derived from the empirical positive correlation revealed in Sec. 4.1: it is more proper to re-estimate the value entries of high uncertainty based on the other ones which are relatively more reliable. **3)** The family of evaluation Q -matrix reconstruction (**-E**) performs comparably to the family of target Q -matrix reconstruction (**-T**): the former performs better in HalfCheetah and Hopper, while the latter

Table 2. Evaluation results of each algorithm (refer to Tab. 1) in four MuJoCo environments. The results are means and stds of average return after 1 million step over 3 trials. Top-1 results in each environment are marked in bold.

Alg. / Env.	Walker2d	HalfCheetah	Ant	Hopper
DDPG	1192±225	7967±439	715±220	1348±339
SVRL-E	1458±238	8039±398	443±221	1690±468
UA-LQE-E-CB	1835±348	5429±1020	421±140	622±129
UA-LQE-E-BB	1716±503	9555±262	707±203	1751±480
SVRL-T	1863±296	9819±392	340±102	1721±422
UA-LQE-T-CB	2430±506	6764±346	400±194	1029±202
UA-LQE-T-BB	2477±549	8727±697	1182±568	823±221

is better in Walker2d and Ant. We hypothesize that this is because two families influence the learning process in different ways (i.e., self-consistency regularization v.s. error reduction in approximation target, as discussed in Sec. 3.2). There remains the possibility to combine of these two orthogonal ways for better results, and we leave the study on such algorithms in the future. **4)** For two kinds of uncertainty quantification methods, bootstrapped-based (**BB**) variants show superiority over count-based method (**CB**) in an overall view. This is consistent to the popularity of bootstrapped-based uncertainty quantification in RL exploration researches [18,19]. Notably, among all variants, UA-LQE-T-BB achieves significant improvement in Walker2d and Ant, which are of more complex kinematics among the four environments. This reveals the potential of UA-LQE in improving the performance of DRL agents.

Finally, it remains much space for improving and developing our proposed algorithm UA-LQE. Our results in MuJoCo is preliminary and obtained without much hyperparameter tuning. A fine-grained hyperparameter tuning is expected to further render the effectiveness of UA-LQE. Moreover, it remains space to improve uncertainty quantification with more accurate model and matrix reconstruction with more efficient algorithm. We leave these aspects for future work.

6 Conclusion

In this paper, we demonstrate the existence of low-rank structure of value function of DRL agents in continuous control problem and introduce two ways of performing Q -matrix reconstruction for value function learning with continuous action space. Moreover, we are the first to empirically reveal a positive correlation between the rank of value matrix and value estimation uncertainty. Based on this, we further propose a novel Uncertainty-aware Low-rank Q -matrix Estimation (UA-LQE) algorithm, whose effectiveness is demonstrated by DDPG-based implementation in several MuJoCo environments. Our work provides some first-step exploration on leveraging underlying structure in value function to improve DRL, which can be further studied, e.g., with more advanced and efficient matrix estimation techniques and other re-estimation schemes in the future.

Acknowledgement

The work is supported by the National Natural Science Foundation of China (Grant No. 62106172)

Appendix

A More Background on Matrix Estimation

Matrix estimation is mainly divided into two types, matrix completion and matrix restoration. In this paper, we focus on the former. We use matrix estimation and matrix completion alternatively. Consider a low-rank matrix M , in which the values of some entries are missing or unknown. The goal of matrix completion problem is to recover the matrix by leveraging the low-rank structure. We use Ω to denote the set of subscripts of known entries and the use $\hat{M}$ to denote the recovered (also called reconstructed in this paper) matrix. Formally, the matrix completion problem is formalized as follows:

$$\begin{aligned} \min \quad & \|\hat{M}\|_*, \\ \text{subject to} \quad & \hat{M}_{i,j} = M_{i,j}, \quad \forall (i,j) \in \Omega \end{aligned} \quad (5)$$

Here $\|\cdot\|_*$ is nuclear norm. The purpose of the above optimization problem is to fill the original matrix as much as possible while keeping the rank of the filled matrix $\hat{M}$ low.

Some representative methods to solve this optimization problem are Fix Point Continuation (FPC) algorithm [14], Singular Value Thresholding (SVT) algorithm [2], OptSpace algorithm [9], Soft-Impute algorithm [15] and so on.

In this paper, we use Soft-Impute algorithm for matrix completion, i.e., Q -matrix reconstruction. A pseudocode for Soft-Impute is shown in Algorithm 2. Define function P_Ω as below:

$$P_\Omega(M)(i,j) = \begin{cases} M_{ij}, & \text{if } (i,j) \in \Omega \\ 0, & \text{if } (i,j) \notin \Omega \end{cases} \quad (6)$$

Similar, we define function $P_{\bar{\Omega}}$. Soft-Impute algorithm completes the missing entries in $\bar{\Omega}$ in an iterative reconstruction fashion. During each iteration, the singular matrix S is obtained by SVD decomposition, and then a filtering is performed through subtracting the value λ controlled by a filtering divider hyperparameter ζ ; next, the new matrix M_{new} is reconstructed with the subtracted singular matrix S_λ . Afterwards, an iteration termination check is performed by comparing whether the difference between the matrices before and after the iteration exceeds the termination threshold ϵ . When the termination threshold is met or the max iteration number is reached, the algorithm finally outputs the completed matrix $\hat{M}$.

Algorithm 2: Soft-Impute Algorithm

```

1 Input: Raw matrix  $M$ , the set of the subscripts of known entries  $\Omega$  and the set
  of the subscripts of unknown (or missing) entries  $\bar{\Omega}$ 
2 Reset the value of the entries in  $\Omega$  of  $M$  with 0 and initialize  $M_{old} = M$ 
3 Initialize singular filtering divider  $\zeta$  and iteration termination threshold  $\epsilon$ 
4 for  $t \leftarrow 1$  to Max Iteration Number do
5   // matrix decomposition, singular filtering and matrix reconstruction
6    $U \times S \times V = \text{SVD}(P_{\Omega}(M_{old}) + P_{\bar{\Omega}}(M))$ 
7   //  $S$  is the diagonal matrix with singular values  $\{s_i\}$  on the diagonal
8    $\lambda = \max_i s_i / \zeta$ 
9    $S_{\lambda} = S - \lambda * E$ , where  $E$  is the identity matrix
10   $M_{new} = U \times S_{\lambda} \times V$ 
11  // check the termination condition
12  if  $\frac{|M_{new} - M_{old}|_F^2}{|M_{old}|_F^2} > \epsilon$  then
13    Set  $M_{old} = M_{new}$ 
14  else
15    Set Set  $\hat{M} = M_{new}$ 
16    break;
17 Output  $\hat{M}$ 

```

B Additional Experimental Details

All regular hyperparameters and implementation details are described in the main body of the paper.

For code-level details, our codes are implemented with Python 3.6.9 and Torch 1.3.1. All experiments were run on a single NVIDIA GeForce GTX 1660Ti GPU. Our Q -matrix reconstruction algorithm is implemented with reference to <https://github.com/YyzHarry/SV-RL>.

For matrix reconstruction algorithm, i.e., SoftImpute, we set the filtering divider $\zeta = 50$, termination threshold $\epsilon = 0.0001$ and Max Iteration Number to be 100.

C Complete Learning Curves of Tab. 2

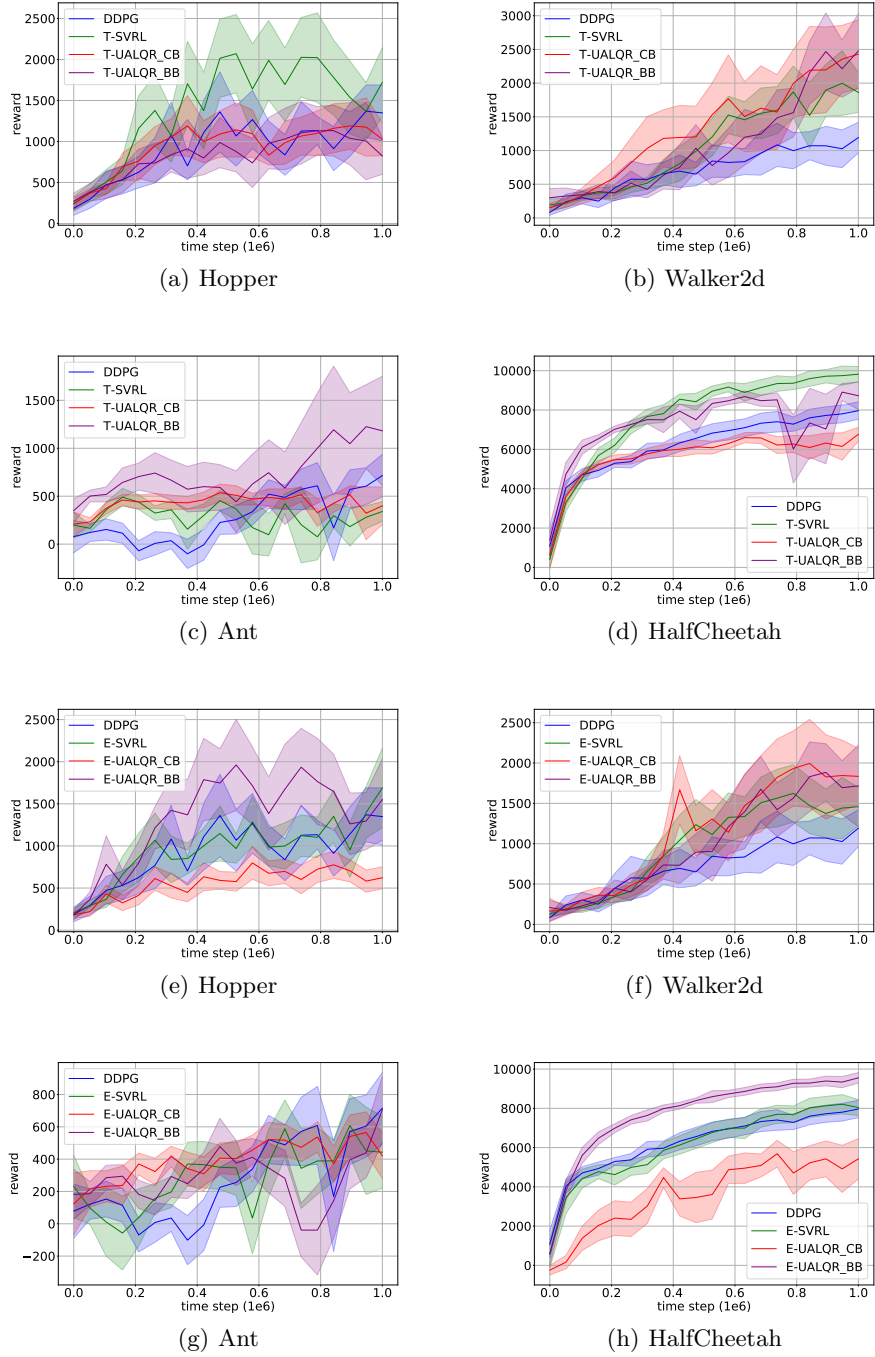


Fig. 6. For the reward curve, the first four are the results of reconstruction on the target Q -matrix, and the last four are the results of reconstruction on the current Q -matrix. Results are the means and stds over 3 trials.

References

1. Bellemare, M., Srinivasan, S., Ostrovski, G., Schaul, T., Saxton, D., Munos, R.: Unifying count-based exploration and intrinsic motivation. In: NeurIPS. pp. 1471–1479 (2016)
2. Cai, J., Candès, E.J., Shen, Z.: A singular value thresholding algorithm for matrix completion. *SIAM J. Optim.* **20**(4), 1956–1982 (2010)
3. Ciosek, K., Vuong, Q., Loftin, R., Hofmann, K.: Better exploration with optimistic actor critic. In: NeurIPS. pp. 1785–1796 (2019)
4. Fujimoto, S., v. Hoof, H., Meger, D.: Addressing function approximation error in actor-critic methods. In: ICML (2018)
5. Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: ICML. pp. 1856–1865 (2018)
6. Hafner, D., Lillicrap, T.P., Ba, J., Norouzi, M.: Dream to control: Learning behaviors by latent imagination. In: ICLR (2020)
7. Hasselt, H., Doron, Y., Strub, F., Hessel, M., Sonnerat, N., Modayil, J.: Deep reinforcement learning and the deadly triad. *CoRR* **abs/1812.02648** (2018)
8. He, J., Zhou, D., Gu, Q.: Uniform-pac bounds for reinforcement learning with linear function approximation. *CoRR* **abs/2106.11612** (2021)
9. Keshavan, R.H., Oh, S., Montanari, A.: Matrix completion from a few entries. In: IEEE International Symposium on Information Theory, ISIT 2009, June 28 - July 3, 2009, Seoul, Korea, Proceedings. pp. 324–328. IEEE (2009)
10. Kumar, A., Agarwal, R., Ghosh, D., Levine, S.: Implicit under-parameterization inhibits data-efficient deep reinforcement learning. In: ICLR (2021)
11. Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D.: Continuous control with deep reinforcement learning. In: ICLR (2015)
12. Luo, X., Meng, Q., He, D., Chen, W., Wang, Y.: I4R: promoting deep reinforcement learning by the indicator for expressive representations. In: IJCAI. pp. 2669–2675. ijcai.org (2020)
13. Lyle, C., Rowland, M., Ostrovski, G., Dabney, W.: On the effect of auxiliary tasks on representation dynamics. In: AISTATS. vol. 130, pp. 1–9 (2021)
14. Ma, S., Goldfarb, D., Chen, L.: Fixed point and bregman iterative methods for matrix rank minimization. *Math. Program.* **128**(1-2), 321–353 (2011)
15. Mazumder, R., Hastie, T., Tibshirani, R.: Spectral regularization algorithms for learning large incomplete matrices. *J. Mach. Learn. Res.* **11**, 2287–2322 (2010)
16. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M.A., Fidjeland, A., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D.: Human-level control through deep reinforcement learning. *Nature* **518**(7540), 529–533 (2015)
17. Ong, H.: Value function approximation via low-rank models. *CoRR* **abs/1509.00061** (2015)
18. Osband, I., Blundell, C., Pritzel, A., Roy, B.V.: Deep exploration via bootstrapped DQN. In: Lee, D.D., Sugiyama, M., von Luxburg, U., Guyon, I., Garnett, R. (eds.) *Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, December 5-10, 2016, Barcelona, Spain.* pp. 4026–4034 (2016)
19. Pathak, D., Gandhi, D., Gupta, A.: Self-supervised exploration via disagreement. In: ICML. vol. 97, pp. 5062–5071 (2019)

20. Scherrer, B., Ghavamzadeh, M., Gabillon, V., Lesner, B., Geist, M.: Approximate modified policy iteration and its application to the game of tetris. *J. Mach. Learn. Res.* **16**, 1629–1676 (2015)
21. Schreck, J.S., Coley, C.W., Bishop, K.J.: Learning retrosynthetic planning through simulated experience. *ACS central science* **5**(6), 970–981 (2019)
22. Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T.P., Leach, M., Kavukcuoglu, K., Graepel, T., Hassabis, D.: Mastering the game of go with deep neural networks and tree search. *Nature* **529**(7587), 484–489 (2016)
23. Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., Riedmiller, M.A.: Deterministic policy gradient algorithms. In: *ICML*. pp. 387–395 (2014)
24. Sutton, R.S., Barto, A.G.: Reinforcement learning: An introduction. *IEEE Transactions on Neural Networks* **16**, 285–286 (1988)
25. Tang, H., Houthoofd, R., Foote, D., Stooke, A., Chen, X., Duan, Y., Schulman, J., Turck, F.D., Abbeel, P.: #exploration: A study of count-based exploration for deep reinforcement learning. In: Guyon, I., von Luxburg, U., Bengio, S., Wallach, H.M., Fergus, R., Vishwanathan, S.V.N., Garnett, R. (eds.) *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, 4-9 December 2017, Long Beach, CA, USA*. pp. 2753–2762 (2017)
26. Vinyals, O., Babuschkin, I., Czarnecki, W.M., Mathieu, M., Dudzik, A., Chung, J., Choi, D.H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J.P., Jaderberg, M., Vezhnevets, A.S., Leblond, R., Pohlen, T., Dalibard, V., Budden, D., Sulsky, Y., Molloy, J., Paine, T.L., Gulcehre, C., Wang, Z., Pfaff, T., Wu, Y., Ring, R., Yogatama, D., Wünsch, D., McKinney, K., Smith, O., Schaul, T., Lillicrap, T., Kavukcuoglu, K., Hassabis, D., Apps, C., Silver, D.: Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature* **575**(7782), 350–354 (2019)
27. Yang, T., Tang, H., Bai, C., Liu, J., Hao, J., Meng, Z., Liu, P.: Exploration in deep reinforcement learning: A comprehensive survey. *CoRR* **abs/2109.06668** (2021)
28. Yang, Y., Zhang, G., Xu, Z., Katabi, D.: Harnessing structures for value-based planning and reinforcement learning. In: *ICLR* (2020)